

Iqra 1

Brick Breaker Game

 Quick Submit Quick Submit National University of Computer and Emerging Sciences, Islamabad

Document Details

Submission ID

trn:oid::1:3420657652

Submission Date

Nov 22, 2025, 9:57 AM GMT+5

Download Date

Nov 22, 2025, 9:58 AM GMT+5

File Name

COAL_Report.docx

File Size

26.4 KB

6 Pages

1,140 Words

6,234 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **0 AI-generated only 0%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Project Report (Fall-2025)
Organization of Computer & Assembly Language
(EL-2003)

Project Title: Brick Breaker Game

Date Of Submission: 22nd November 2025

Group Members:

Syeda Maha Fatima (24K-0624)

Fabiha Siddique (24K-1022)

Iqra (24K-1024)

Summary:

Our Project “**Brick Breaker Game**” is a console-based implementation of x86 Assembly Language using MASM and Irvine32. Its purpose was to successfully implement the logic behind the original Brick Breaker Game using the core concepts of low-level programming. Built from scratch, it includes input handling, console manipulation, sound integration, game loop controlling, usage of registers and much more. The game features a main menu, difficulty levels, level upgrading and a scoring system.

Introduction:

- **Background:**

This project is highly related to COAL as it is solely based on low-level programming and while building it, we used the following concepts:

- Register Manipulation
- Knowledge of Stack & Memory
- Handling Input/Output at low-level
- Programming in Assembly Language (x86)
- Handling Errors & Exceptions

- **Objectives:**

We successfully achieved the goals mentioned in our proposal which were:

- **Memory management:** Includes stack, registers.
- **Input/Output Handling:** Required understanding of low-level programming.
- **Efficient Game Loop Control:** Using loops, jump conditions, cmp operator and counter registers etc.

Project Description:

- **Scope:**

We included various features in our game which include:

- Ball Movement
- Controlled Paddle Movement
- Collisions
- Difficulty Modes (Easy, Medium, Hard)
- 3 Levels with increasing breaking powers (HP) of bricks.
- Scoring

- Bonus Life on reaching to level3
- Game Over & Main-Menu Screen
- Sounds like collisions, navigation, losing lives, level up and game over.

Still there are some features excluded such as:

- Save/Load Feature
- Online players
- More than 3 levels.

• **Technical Overview:**

The tools and technologies used in our project are:

Language: x86Assembly Language
Software: Visual Studio 2022 + Windows Console
Assembler: MASM
Libraries: Irvine32, Macros and Winmm (for sound)

Methodology:

• **Approach:**

Almost, it took us 2weeks for project completion.

Day1-2: Built Welcome Screen & Main Menu.

Day3-4: Drawing Bricks & Paddle

Day5-7: Collision Logic

Day8-9: Added Difficulty Modes & Some Error Handling

Day10-12: Added 3 Levels

Day13-14: Sound Integration & Final Enhancements

• **Roles & Responsibilities:**

- **Syeda Maha Fatima:** Sounds & Color Graphics
- **Fabiha Siddiqui:** Main Menu & Other Screens
- **Iqra:** Core Logic

Project Implementation:

• Design & Structure:

1. Loading Screen
2. Welcome Screen (Asks for player name)
3. Main Menu (Includes Start Game, Instructions, High Score & Exit)
4. Start Game (Asks for choosing between Easy, Medium & Hard)
5. Main Game (Level-up if all bricks destroyed, else game over appears)
6. Winner Screen (If wins the game)
7. Game Over Screen
8. Either Exit or Back to Main Menu

& the loop continues.

• Functionalities:

The following primary functions have been implemented:

- **DrawPaddle** (Draws paddle)
- **ErasePaddle** (Erases paddle)
- **DrawBox** (Used to draw the box for each screen)
- **Screen1** (Handles the welcome screen & entire main menu under one procedure)
- **DrawBall** (Draws Ball)
- **LevelGame** (Used inside screen1 to display the main game screen where the entire logic is being implemented, tracks currentLevel & levelCleared)
- **DrawBlock** (Draws single brick)
- **DrawBlockRows** (Using DrawBlock, draws bricks in three rows)
- **MoveBall** (Tracks wall collisions, brick collisions and paddle collisions, redraws ball accordingly using DrawBall, tracks lives and resets ball's position whenever player loses a life)
- **BrickCollision** (A Separate procedure defined to track bricks power according to the levels defined, also erases brick everytime a brick is destroyed: Used inside **MoveBall**)
- **ReadKeyIfAny** (During the game loop, reads keys [A,D,S,E] A for moving paddle to the left, D for moving paddle to the right, S for pause/resume the game and E for ending the game)
- **PauseBox** (Draws pause box whenever S is pressed to pause the game)
- **ErasePauseScreen** (Erases pause box whenever S is pressed to resume the game)
- **ResetBricks** (Resets bricksStatus array to its defined power according to the currentLevel: For Level1, BrickPower= 1, For Level2, BrickPower= 2, For Level3, BrickPower= 3, while 0 means destroyed brick.)
- **DifficultyScreen** (Used inside **Screen1** for displaying Easy,Hard,Medium)
- **GameOver** (Displays game over screen when all lives are lost)
- **HighScoreScreen** (Used inside **Screen1** for displaying highScore)
- **ShowLevelCompleted** (Displays a box whenever a level is cleared)
- **EraseLevelCompleted** (Erases the levelBox for continuing the next level)
- **WinnerScreen** (If all levels cleared, winner screen appears)

- **Challenges Faced:**

Behind the entire project, there were counterfeit frustrating moments that were quite challenging. Some of them were:

- Screen Size Issue
- The 'loop' operator was a bit disturbing in few places because it couldn't jump more than 127 bytes.
- In 2D Array, the ECX counter for erasing bricks and detecting collision at the same time was also challenging which was later solved by pushing one of them in the stack and popping it.
- Integrating sound was also a bit challenging in Visual Studio.
- The Physics & Calculations of ball, paddle, walls and bricks coordinates also required deep understanding.

Results:

- **Project Outcomes:**

The challenges were worth it, because in the end it resulted in a successful implementation of our Brick Breaker Game. We successfully achieved our goals, and our outcome was a console-based Brick Breaker Game which included the concepts of Assembly Language only.

- **Screenshots & Illustrations:**

- **Testing & Validation:**

With addition of every new functionality, we made sure to evaluate it first before moving forward to another. Firstly, the main menu was evaluated, then the paddle and bricks display was assessed onwards the entire logic by playing the game with every test case: Such as for losing lives, for winning the game and everything else. For each level, we evaluated the game repeatedly to make sure there wasn't any error. We also assessed If the game loop was working properly or not. Testing for the x and y coordinates was also part of our project.

At the end, the final testing included all the features of our game.

Conclusion:

- **Summary of Findings:**

Our project is proof that using assembly & its concepts, such projects can be implemented with console only.

Although, it sometimes becomes difficult to work with limited memory registers, syntax limitations, memory management in low level programming but still it is possible to implement real-time applications with multi-media integration in Assembly.

With further enhancements, this and similar to this many other projects in assembly can stand out among other projects which are made in high level languages. While high level languages are easier to implement, low level languages provide us a deeper understanding of how memory management of computers really works.

- **Final Remarks:**

Developing this game in Assembly exceeded our expectations which is a testament to the capabilities of low-level programming.

In Future, addition of other features can enhance this project which may include Save/Load Feature, Live Scoreboard, Infinite levels, Special Bricks and much more.